

SECUREVISION AI

CIFAR-10 Surveillance Image Classification System

Technical Project Report & Reproducibility Guide

SecureVision AI is an end-to-end machine-learning project that demonstrates how an image classification workflow can be developed from raw data validation through model training, evaluation, inference, operational analytics, and an interactive application. The project uses the CIFAR-10 image dataset as a controlled benchmark for ten visual categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

The project follows a twelve-step lifecycle covering problem definition, data acquisition, data ingestion and sanitization, exploratory analysis, feature preparation, leakage-free validation, baseline and advanced modeling, multi-metric evaluation, error diagnostics, analytics integration, interactive visualization, and operational recommendations.

The implementation is organized as a reproducible Python project with separate modules for data engineering, exploratory analysis, model training, inference, validation, cost/threshold analysis, testing, and application delivery.

Project Objectives

- Build a reproducible CIFAR-10 image-classification pipeline with explicit data-quality and leakage checks.
- Compare a conventional CNN baseline with a ResNet18-based advanced model.
- Evaluate models using accuracy, Macro Precision, Macro Recall, Macro F1, Weighted F1, confusion matrices, and latency.
- Create an inference pipeline that accepts a single image, reloads trained weights, predicts a class, reports confidence, and records latency.
- Connect predictions to operational analytics including error cost, threshold sensitivity, and Type I/Type II error analysis.
- Provide an interactive Streamlit application for prediction, project metrics, and project documentation.

Scope and Interpretation

CIFAR-10 is a small natural-image benchmark rather than a real surveillance-frame dataset. Therefore, SecureVision AI should be interpreted as a technical prototype demonstrating the ML lifecycle and deployment workflow, not as evidence of production surveillance performance.

Primary evaluation metrics: Macro F1 and inference latency, with additional metrics reported to provide a fuller view of classification quality.

1. Dataset, Data Provenance and Data Engineering

1.1 Dataset Description

The project uses CIFAR-10, containing 60,000 color images of size 32×32 pixels distributed across ten classes. The implementation uses the standard 50,000-image training portion and 10,000-image test portion.

Property	Project value
Dataset	CIFAR-10
Training images	50,000
Test images	10,000
Image dimensions	32 × 32 × 3
Classes	10
Training split used	45,000
Validation split	5,000
Held-out test split	10,000
Split method	Stratified, random state 42

1.2 Source and Provenance

The project dataset archive was supplied through the project Google Drive source and stored outside the Git repository because the raw archive is approximately 162.6 MB. The repository contains a reproducible download/extraction script and a small sample rather than the full raw archive.

1.3 Data Quality and Leakage Controls

- Validated image count, shape, dtype, pixel range, labels, and class distribution.
- Checked for missing values and malformed records.
- Performed MD5-based duplicate detection.
- Checked for exact train/test image overlap to identify direct leakage.
- Created a reproducible train/validation/test index file.
- Kept preprocessing consistent between training and inference.

1.4 Preprocessing

Images are represented as (N, 32, 32, 3), converted to float32, and normalized to the [0,1] range before model-specific standardization. The model pipeline uses CIFAR-10 channel means [0.4914, 0.4822, 0.4465] and standard deviations [0.2470, 0.2435, 0.2616].

1.5 Exploratory Data Analysis

- Generated class distribution and image-statistics outputs.
- Calculated brightness, contrast, and RGB-channel statistics.
- Computed class-wise statistics and skewness.
- Applied IQR-based anomaly screening to identify extreme image-level statistics.
- Produced anomaly summaries and top EDA observations for downstream interpretation.

Key engineering principle: preprocessing, splitting, training, evaluation, and inference are implemented as reproducible steps rather than relying on undocumented notebook state.

2. Machine Learning Methodology

2.1 Modeling Strategy

SecureVision AI compares two convolutional approaches. The baseline CNN provides a relatively simple reference model, while ResNet18 provides a deeper residual architecture intended to capture more useful visual representations.

Configuration	Baseline CNN	ResNet18
Architecture	Custom CNN	Residual CNN / ResNet18
Task	10-class classification	10-class classification
Epochs	15	15
Batch size	128	128
Learning rate	0.001	0.001
Augmentation	Training augmentation	Training augmentation
Standardization	CIFAR-10 mean/std	CIFAR-10 mean/std
Output	10-class logits	10-class logits

2.2 Validation Strategy

The data is separated into 45,000 training images, 5,000 validation images, and 10,000 held-out test images using stratification and random state 42. Training uses only the training partition; the validation partition supports development checks, while the test partition is reserved for held-out performance reporting.

2.3 Training and Serialization

- Training is implemented in `src/model.py`.
- The project uses TensorFlow/Keras for model construction and optimization.
- Model weights are saved as `baseline_cnn.weights.h5` and `resnet18.weights.h5`.
- The weights are below the repository's 50 MB large-file limit.
- The inference module reconstructs the architecture before loading the saved weights.

2.4 Held-Out Test Results

Metric	Baseline CNN	ResNet18
Accuracy	0.7975	0.9041
Macro Precision	0.7969	0.9039
Macro Recall	0.7975	0.9041
Macro F1	0.7970	0.9039
Weighted F1	0.7970	0.9039

On the held-out test set, ResNet18 produced higher values for every reported classification metric. Its Accuracy and Macro F1 were approximately 0.904, compared with approximately 0.798 for the baseline CNN.

2.5 Why Multiple Metrics?

Accuracy gives the overall proportion of correct predictions, while Macro Precision, Macro Recall, and Macro F1 treat the ten classes more evenly by calculating class-level performance and averaging across classes. Weighted F1 accounts for class support. Together these metrics provide a more useful view than accuracy alone.

3. Evaluation, Error Analysis and Operational Analytics

3.1 Multi-Metric Evaluation

The validation pipeline generates summary metrics, per-class metrics, confusion matrices, and prediction records for both models. The batch prediction output contains the true class, predicted class, confidence, correctness, and per-class probabilities.

3.2 Confusion and Error Diagnostics

- Confusion matrices are generated for both validation and test partitions.
- Per-class precision, recall, F1, and support are retained for class-level diagnosis.
- Prediction records make individual errors traceable back to their sample index and model.
- The analysis is designed to identify classes that are systematically confused rather than relying only on a single aggregate score.

3.3 Cost Matrix Analysis

The analytics layer converts classification outcomes into an explicit cost framework. The default illustrative configuration assigns zero cost to a correct prediction and one unit to an incorrect prediction. The project also generates cost-by-class, confusion-cost, and summary outputs so that alternative operational costs can be substituted without changing the prediction pipeline.

3.4 Threshold and Review Analysis

The decision module evaluates confidence thresholds and produces threshold sweeps, cost sensitivity, and Type I/Type II error summaries. A threshold-based workflow can accept high-confidence predictions automatically while routing lower-confidence predictions for review. The outputs include coverage, review rate, accepted accuracy, errors caught by review, total cost, and cost per 1,000 cases.

3.5 Type I and Type II Considerations

For a multiclass classification workflow, the practical meaning of an error depends on the operational action associated with each class. The project therefore reports class-level error behavior rather than assuming that every misclassification has identical consequences.

3.6 Inference and Latency

- `src/predict.py` loads either trained architecture and its saved weights.
- An input image is validated to the required 32×32×3 shape.
- The image is normalized and standardized using the same model preprocessing.
- A warm-up inference is performed before latency measurement.
- Softmax probabilities provide predicted class and confidence.
- Inference latency is logged to `data/processed/prediction_log.csv`.

Operational note: classification quality and inference latency are separate concerns. A model can improve predictive metrics while introducing additional computational cost, so both dimensions should be considered when moving beyond a prototype.

4. Application, Dashboard and Repository Guide

4.1 Interactive Application

The Streamlit application provides a practical front end for the project. The application is organized around three areas: image prediction, project metrics, and project information.

Application area	Function
Predict	Upload a 32×32-compatible image, run the selected model, display the predicted class, confidence, and latency.
Project Metrics	Display model comparison, class-level results, confusion/error analysis, cost and threshold outputs, and latency information.
About Project	Explain the problem, dataset, methodology, project scope, and implementation workflow.

4.2 End-to-End Workflow

Raw CIFAR-10 data → validation and quality audit → stratified split → preprocessing and EDA → CNN/ResNet18 training → serialized model weights → batch evaluation → cost/threshold analytics → single-image inference → Streamlit presentation.

4.3 Repository Structure

```
README.md
requirements.txt
.gitignore
data/raw/
data/processed/
notebooks/ML_Project.ipynb
src/preprocessing.py
src/model.py
src/predict.py
src/eda.py
src/validations/
models/
dashboard/
tests/
docs/
report/
presentation/
```

4.4 Main Components

Component	Purpose
preprocessing.py	Data loading, validation, splitting, normalization, and reproducible preprocessing.
eda.py	Dataset statistics, distributions, skewness, and anomaly analysis.
model.py	Baseline CNN and ResNet18 construction and training.
predict.py	Reusable single-image inference, confidence, latency, and logging.
batch_predictor.py	Batch predictions for validation/test analysis.
derived_metrics.py	Accuracy, Macro Precision/Recall/F1 and related metrics.
cost_matrix.py	Confusion-cost and error-cost analysis.
decision.py	Confidence threshold, review, cost, and Type I/II analysis.
app.py	Interactive Streamlit application.

4.5 Reproducibility and Large Files

- The raw 162.6 MB archive is not committed to GitHub.
- `data/raw/download_data.py` provides reproducible acquisition and extraction.
- A 500-row/sample metadata artifact is retained for repository inspection.
- Generated model weights used by the application are kept below the project's 50 MB file limit.
- `requirements.txt` pins the main Python package versions used by the project.

5. Reproduction Guide, Limitations and Project Conclusion

5.1 Environment Setup

The project was developed with Python 3.11 and a project-local virtual environment. The core dependencies include NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, TensorFlow, Jupyter, ipykernel, Pillow, and gdown.

Typical setup commands

```
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
python -m src.model
python -m src.predict
streamlit run app.py
```

The exact training command can be adjusted according to available compute. The project configuration used 15 epochs, batch size 128, and learning rate 0.001 for the documented comparison.

5.2 Testing and Validation

- Run preprocessing tests in tests/test_preprocessing.py.
- Run inference sanity tests in tests/test_inference.py.
- Compile/import checks were used during development to catch module and path errors.
- Held-out test metrics are generated from the stored prediction/evaluation outputs.

5.3 Team Workflow

Role	Primary contribution
Data Engineer	Acquisition, validation, cleaning, leakage checks, reproducible splits and data-quality outputs.
Data Analyst	EDA, distributions, image statistics, anomaly detection and key observations.
Data Scientist	Baseline/advanced model strategy, training, tuning and serialized weights.
ML Engineer	Reusable inference pipeline, model reload, latency measurement, logging and sanity tests.
Analytics Engineer	Derived metrics, cost matrices, threshold analysis, and Type I/II error analysis.
BI / Power BI Developer	Interactive dashboard/application presentation, KPI views and user-facing visualization.

5.4 Current Limitations

- CIFAR-10 contains low-resolution generic object images and does not represent real surveillance-camera conditions.
- The ten CIFAR-10 labels are not security-event categories.
- Real deployments would require domain-specific surveillance data and operational validation.
- Latency depends on hardware, TensorFlow configuration, model loading, warm-up behavior, and input pipeline overhead.
- Threshold and cost analysis is illustrative unless organization-specific costs and review policies are supplied.
- The Streamlit application is a prototype interface rather than a hardened production service.

5.5 Future Improvements

- Train or fine-tune on a domain-specific surveillance dataset with clearly defined operational classes.
- Add stronger calibration and probability-quality analysis.
- Evaluate robustness under blur, lighting changes, compression, occlusion, and camera-domain shift.
- Add experiment tracking, automated CI tests, model/data versioning, and production telemetry.
- Extend monitoring to data drift, confidence drift, class-frequency changes, and inference failures.

5.6 Conclusion

SecureVision AI demonstrates a complete, reproducible machine-learning lifecycle rather than only a model-training notebook. The baseline CNN achieved 0.7975 held-out accuracy and 0.7970 Macro F1, while ResNet18 achieved 0.9041 held-out accuracy and 0.9039 Macro F1. These results establish a clear model-performance difference on the CIFAR-10 benchmark used in the project. The surrounding data-quality checks, evaluation outputs, inference pipeline, latency logging, operational

cost analysis, and Streamlit interface make the project suitable as an end-to-end ML engineering demonstration.

The GitHub repository contains the implementation, reproducibility files, tests, generated evaluation outputs, model weights, and application components. For any real surveillance use case, the next stage would be domain-specific data collection and validation rather than treating CIFAR-10 performance as production evidence.

References

1. CIFAR-10 dataset: Krizhevsky, A. and Hinton, G., Learning Multiple Layers of Features from Tiny Images.
2. He, K. et al., Deep Residual Learning for Image Recognition, CVPR 2016.
3. TensorFlow / Keras documentation for model construction and inference.
4. scikit-learn documentation for evaluation utilities and metrics.
5. Streamlit documentation for interactive Python application development.
6. Project repository: github.com/Preetham-1064/secure-vision-ai